🐆 LUKASH ($LUKA) - ANÁLISIS TÉCNICO COMPLETO Y PROYECCIONES FINANCIERAS

DOCUMENTO MAESTRO DE ARQUITECTURA BLOCKCHAIN Y PLAN FINANCIERO

Versión: 2.0
Fecha: Enero 28, 2026
Elaborado por: Experto en Blockchain & Tokenomics Solana

TABLA DE CONTENIDOS

---

1. RESUMEN EJECUTIVO

1.1 Concepto del Proyecto

LUKASH es un ecosistema financiero descentralizado basado en Solana que opera bajo el **Estándar KASH** (Reserva Fraccionaria Inversa). A diferencia de los sistemas tradicionales que emiten deuda, LUKASH reduce sistemáticamente su supply de tokens mientras construye un vault de activos duros (BTC, SOL, USDC, LINK).

Propuesta de Valor Única:

- **Deflación Sistemática:** Quema de tokens en Etapas 1 y 2
- **Respaldo Real:** 70% del capital inicial va directo a activos KASH
- **Super-App Financiera:** Ecosistema completo de servicios en $LUKA
- **Modelo de Doble Motor:** Fees transaccionales + fees de servicio

1.2 Supply Tokenómico Inicial

Categoría	Tokens	Porcentaje	Destino
Seed/Public Sale	450,000,000	45%	Generación de capital
Liquidity Pool	300,000,000	30%	LP bloqueado/quemado
Marketing/CEX	100,000,000	10%	Campañas y listings
Airdrops/Comunidad	100,000,000	10%	Incentivos usuarios
Staking Reserve	50,000,000	5%	Recompensas staking
TOTAL INICIAL	**1,000,000,000**	**100%**	-

1.3 Distribución del Capital Genesis (Split Fundacional)

Del 100% del capital recaudado en preventa:

- **30%** → LP Injection (pareado con 300M $LUKA y quemado)
- **40%** → Vault KASH Core (inamovible, convertido a BTC/SOL/USDC/LINK)
- **30%** → Vault KASH Society (bloqueado bajo Jaguar Lock)

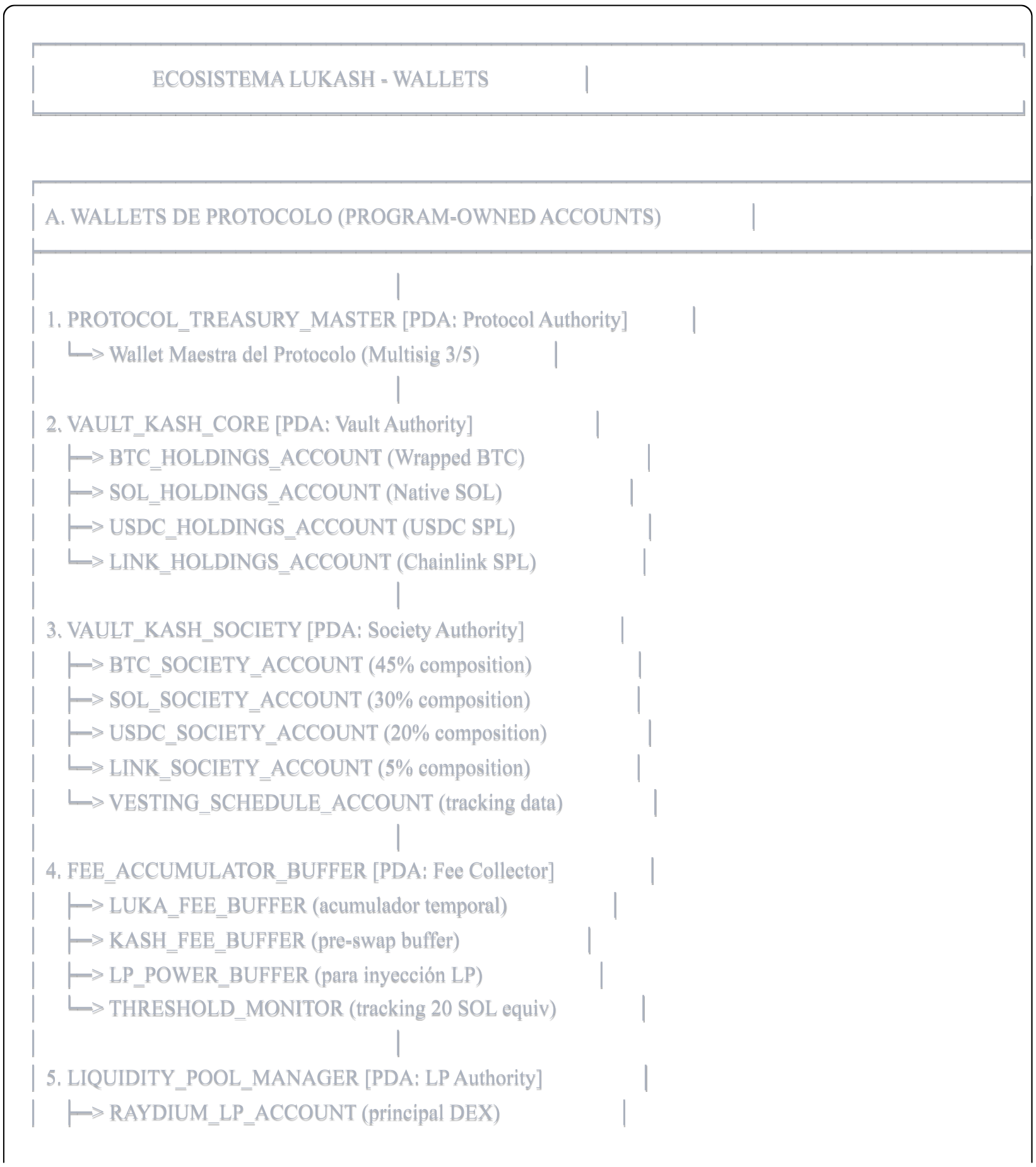
Composición del Vault KASH:

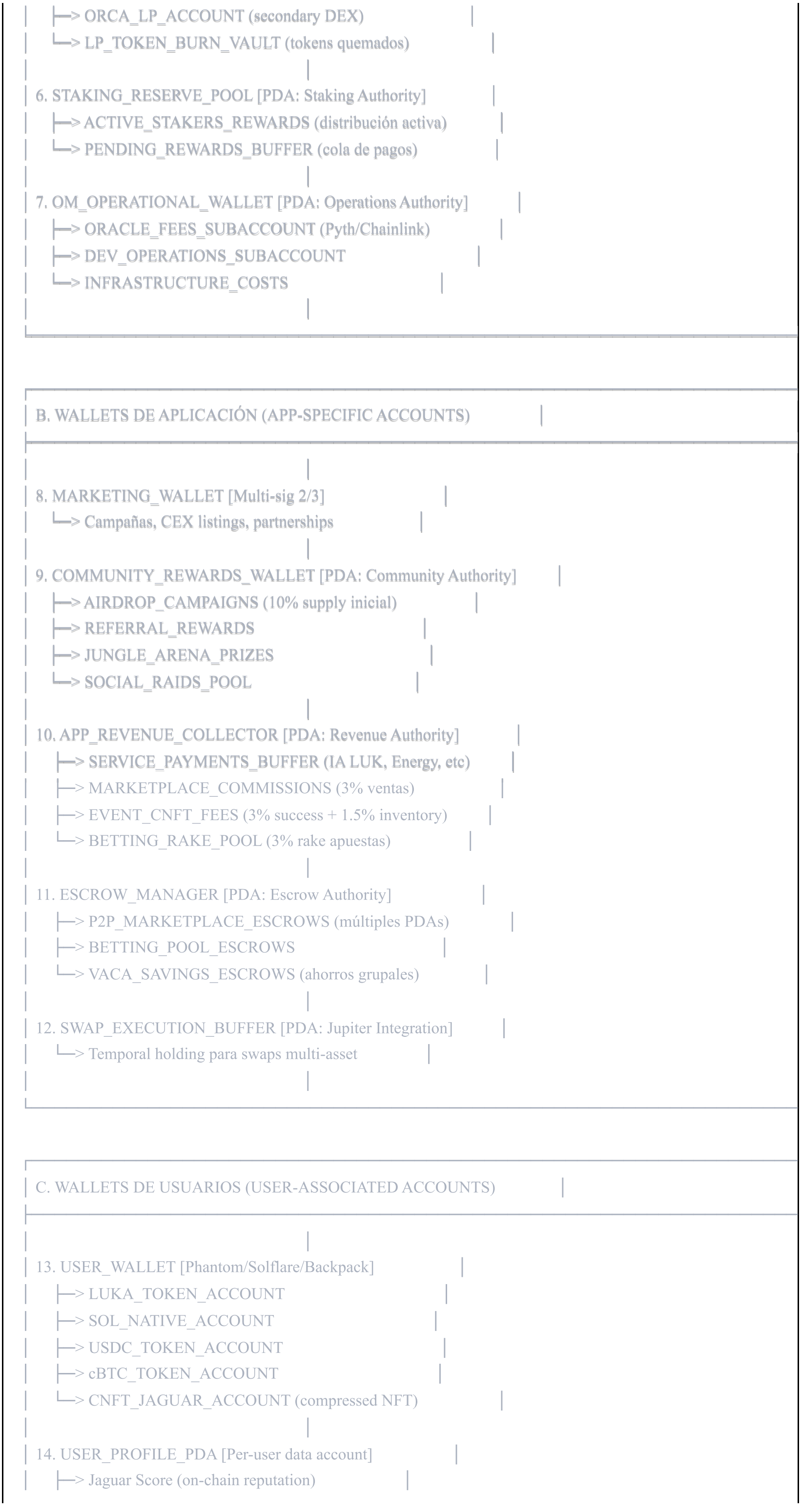
- 45% Bitcoin (BTC)
- 30% Solana (SOL)
- 20% USDC (stablecoin)
- 5% Chainlink (LINK)

---

2. ARQUITECTURA DE WALLETS Y FLUJOS TRANSACCIONALES

2.1 Mapa de Wallets del Ecosistema




├─> ORCA_LP_ACCOUNT (secondary DEX)
└─> LP_TOKEN_BURN_VAULT (tokens quemados)
6. STAKING_RESERVE_POOL [PDA: Staking Authority]
├─> ACTIVE_STAKERS_REWARDS (distribución activa)
└─> PENDING_REWARDS_BUFFER (cola de pagos)
7. OM_OPERATIONAL_WALLET [PDA: Operations Authority]
├─> ORACLE_FEES_SUBACCOUNT (Pyth/Chainlink)
├─> DEV_OPERATIONS_SUBACCOUNT
└─> INFRASTRUCTURE_COSTS
B. WALLETS DE APLICACIÓN (APP-SPECIFIC ACCOUNTS)
8. MARKETING_WALLET [Multi-sig 2/3]
└─> Campañas, CEX listings, partnerships
9. COMMUNITY_REWARDS_WALLET [PDA: Community Authority]
├─> AIRDROP_CAMPAIGNS (10% supply inicial)
├─> REFERRAL_REWARDS
├─> JUNGLE_ARENA_PRIZES
└─> SOCIAL_RAIDS_POOL
10. APP_REVENUE_COLLECTOR [PDA: Revenue Authority]
├─> SERVICE_PAYMENTS_BUFFER (IA LUK, Energy, etc)
├─> MARKETPLACE_COMMISSIONS (3% ventas)
├─> EVENT_CNFT_FEES (3% success + 1.5% inventory)
└─> BETTING_RAKE_POOL (3% rake apuestas)
11. ESCROW_MANAGER [PDA: Escrow Authority]
├─> P2P_MARKETPLACE_ESCROWS (múltiples PDAs)
├─> BETTING_POOL_ESCROWS
└─> VACA_SAVINGS_ESCROWS (ahorros grupales)
12. SWAP_EXECUTION_BUFFER [PDA: Jupiter Integration]
└─> Temporal holding para swaps multi-asset
C. WALLETS DE USUARIOS (USER-ASSOCIATED ACCOUNTS)
13. USER_WALLET [Phantom/Solflare/Backpack]
├─> LUKA_TOKEN_ACCOUNT
├─> SOL_NATIVE_ACCOUNT
├─> USDC_TOKEN_ACCOUNT
├─> cBTC_TOKEN_ACCOUNT
└─> CNFT_JAGUAR_ACCOUNT (compressed NFT)
14. USER_PROFILE_PDA [Per-user data account]
├─> Jaguar Score (on-chain reputation)

2.2 Flujos Transaccionales Detallados

FLUJO 1: Transferencia P2P Standard (Usuario A → Usuario B)

FLUJO: Transferencia P2P de $LUKA

```
[USER_A_WALLET]
    |
    | (1) Instrucción: Transfer 100 $LUKA to USER_B
    ↓
[TRANSFER_HOOK_PROGRAM]
    |
    | (2) Check: Protocol Stage (1, 2 o 3)
    |     └─> Etapa 1: Fee = 5.5%
    |     └─> Etapa 2: Fee = 3.5%
    |     └─> Etapa 3: Fee = 1.5%
    |
    | (3) Check: Anti-Whale Tax
    |     └─> IF amount > 0.5% circulating supply
    |         THEN Fee = 20% (penalty)
    |
    | (4) Check: Anti-Bot Cooldown
    |     └─> IF last_tx_timestamp < 60 seconds
    |         THEN REJECT transaction
    |
    ↓
[FEE_CALCULATION_ENGINE]
    |
    | Ejemplo Etapa 1 (5.5% fee):
    | ──────────────────────────────────
    | Monto enviado: 100 $LUKA
    | Fee total: 5.5 $LUKA
    |   ├─> 2.5% → KASH (2.5 $LUKA)
    |   ├─> 1.0% → LP-POWER (1.0 $LUKA)
    |   ├─> 1.0% → Staking (1.0 $LUKA)
    |   └─> 1.0% → O&M (1.0 $LUKA)
    |
    | Usuario B recibe: 94.5 $LUKA
    |
    ↓
[FEE_DISTRIBUTION_ROUTER]
    |
    ├─> (2.5 $LUKA) → [FEE_ACCUMULATOR_BUFFER].KASH_FEE_BUFFER
    ├─> (1.0 $LUKA) → [FEE_ACCUMULATOR_BUFFER].LP_POWER_BUFFER
    ├─> (1.0 $LUKA) → [STAKING_RESERVE_POOL]
    └─> (1.0 $LUKA) → [OM_OPERATIONAL_WALLET]
    |
    ↓
[THRESHOLD_MONITOR]
    |
    | Check: KASH_FEE_BUFFER >= 20 SOL equivalent?
    |   ├─> NO  → Accumulate and wait
    |   └─> YES → Trigger BATCH_PROCESSOR
    |
    ↓
[BATCH_PROCESSOR] (when threshold met)
    |
```

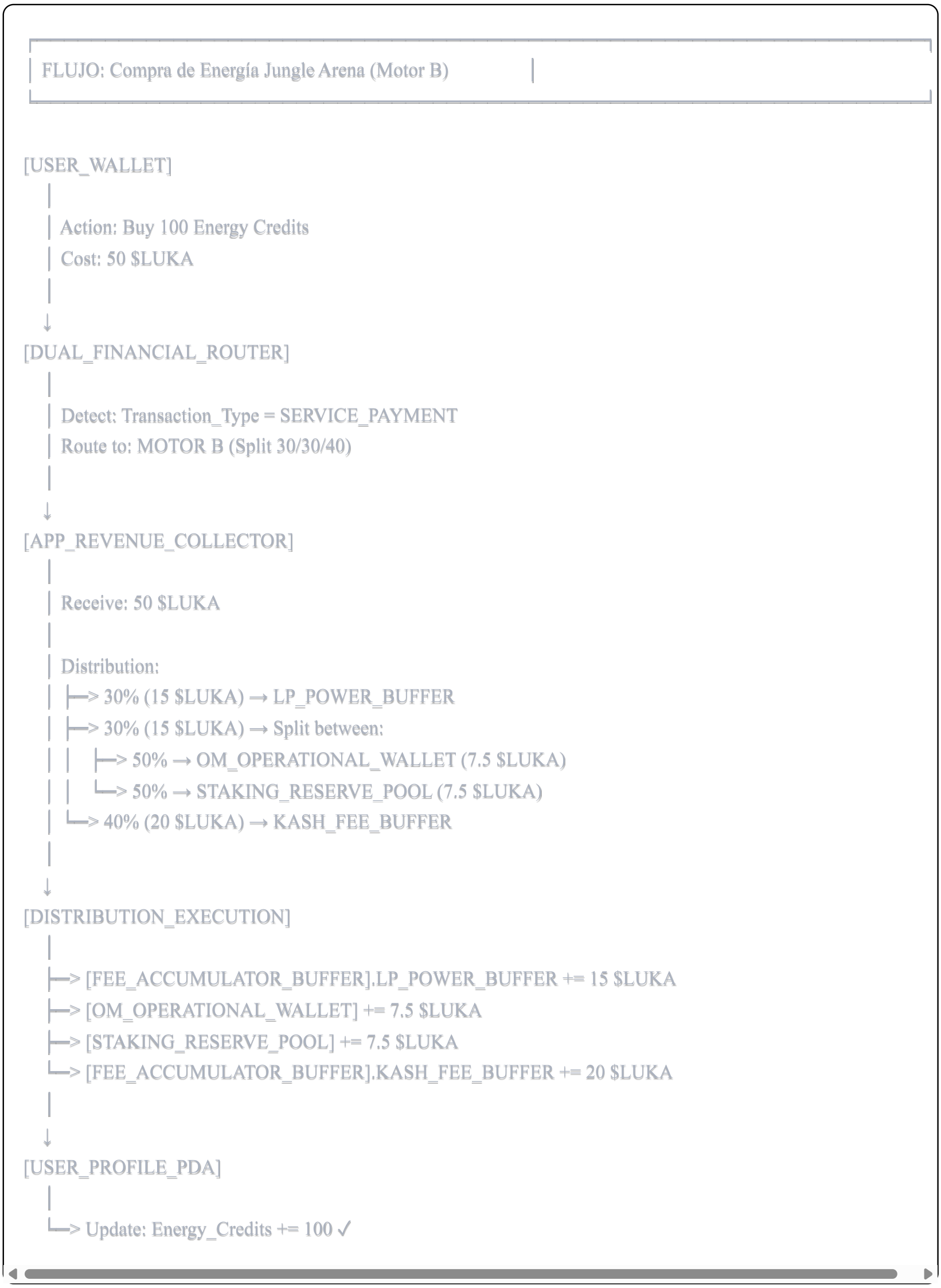
```
    │ IF Stage == 1 or Stage == 2:
    │   └─> BURN accumulated $LUKA in KASH_FEE_BUFFER
    │
    │ IF Stage == 3:
    │   ├─> (1) Swap 50% to SOL via Jupiter (TWAP)
    │   └─> (2) Distribute 30/40/30:
    │       ├─> 30% → Add Liquidity (LP injection)
    │       ├─> 40% → Swap to KASH basket → VAULT_KASH_CORE
    │       └─> 30% → Swap to KASH basket → VAULT_KASH_SOCIETY
    │
    ↓
[USER_B_WALLET]
    │
    └─> Receives: 94.5 $LUKA ✓
```

FLUJO 2: Swap Multi-Asset con Doble Captura (SOL → $LUKA → Payment)

```
FLUJO: Pago con SOL en LUKASH App (Doble Captura)

[USER_WALLET]
    | Balance: 10 SOL
    | Intención: Pagar 5 SOL a MERCHANT_X
    |
    ↓
[LUKASH_APP_GATEWAY]
    |
    | (1) Detect: Input Asset = SOL (not $LUKA)
    |     └─> Activate DUAL_FINANCIAL_ROUTER
    |
    ↓
[CAPA 1: SWAP EXECUTION]
    |
    | (2) Calculate Pre-Swap Fee (Etapa 1: 5.5%)
    |     Amount: 5 SOL
    |     Fee Layer 1: 0.275 SOL
    |     Net to swap: 4.725 SOL
    |
    ├─> Send 0.275 SOL → [SWAP_EXECUTION_BUFFER]
    |   └─> Queue for conversion to $LUKA → FEE_ACCUMULATOR
    |
    └─> Execute Jupiter Swap:
      4.725 SOL → X $LUKA (market rate)
    |
    ↓
[CAPA 2: TRANSFER EXECUTION]
    |
    | (3) Now transfer X $LUKA to MERCHANT_X
    |     Apply Stage Fee again (5.5%)
    |
    |     Example: 4.725 SOL = 1,000 $LUKA
    |     Fee Layer 2: 55 $LUKA
    |
    |     Merchant receives: 945 $LUKA
    |
    ├─> Send 55 $LUKA → [FEE_ACCUMULATOR_BUFFER]
    |   └─> Distribute per stage rules
    |
    └─> Transfer 945 $LUKA → [MERCHANT_WALLET]
    |
    ↓
[RESULTADO FINAL]
    |
    ├─> User paid: 5 SOL
    ├─> Protocol captured:
    |   ├─> Layer 1: 0.275 SOL (converted to LUKA)
    |   └─> Layer 2: 55 $LUKA
    ├─> Merchant received: 945 $LUKA
    |
    └─> Total fee capture: ~11% equivalent (double layer) ✓
```

FLUJO 3: Pagos de Servicios App (Split 30/30/40)



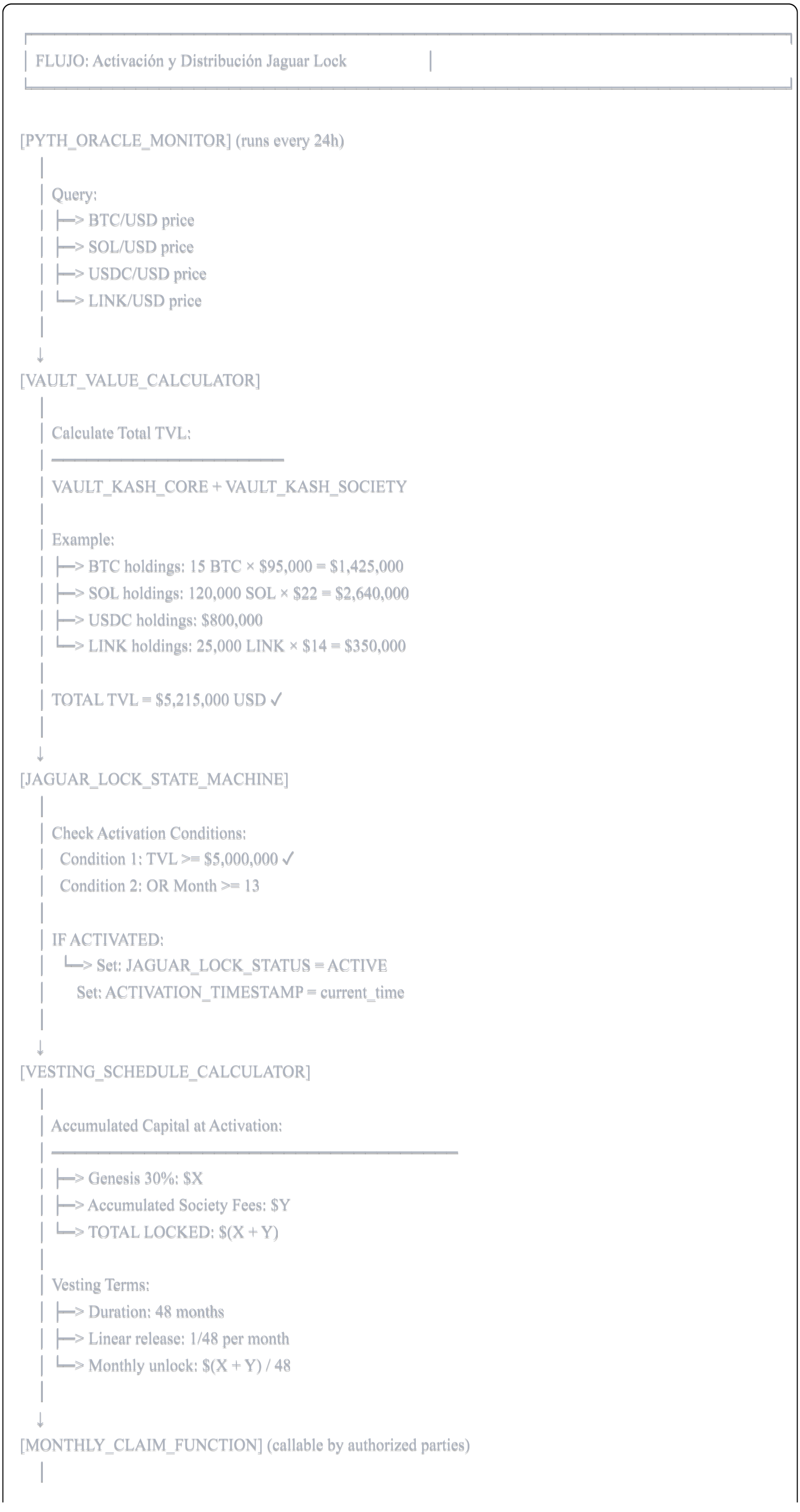
FLUJO 4: Marketplace P2P con Escrow

```
┌──────────────────────────────────────────────────────────────┐
│ FLUJO: Venta Producto P2P (Comisión 3% + Fee Etapa)          │
└──────────────────────────────────────────────────────────────┘

[SELLER_WALLET]
    │ Lists: iPhone 15 Pro
    │ Price: 1,000 $LUKA
    │
    ↓
[BUYER_WALLET]
    │ Clicks: Buy Now
    │
    ↓
[MARKETPLACE_SMART_CONTRACT]
    │
    │ (1) Create Escrow PDA
    │     └─> Seeds: [buyer_pubkey, seller_pubkey, listing_id]
    │
    │ (2) Transfer from Buyer to Escrow
    │     Amount: 1,000 $LUKA
    │     Apply Stage Fee (5.5%): 55 $LUKA
    │
    │     To Escrow: 945 $LUKA
    │     To Fee Buffer: 55 $LUKA
    │
    ↓
[ESCROW_PDA]
    │ Holds: 945 $LUKA
    │ Status: PENDING_DELIVERY
    │
    │ ... (physical delivery happens) ...
    │
    │ Buyer confirms: DELIVERED ✓
    │
    ↓
[RELEASE_FROM_ESCROW]
    │
    │ Calculate Marketplace Commission (3%)
    │   Base: 945 $LUKA in escrow
    │   Commission: 28.35 $LUKA
    │   Seller receives: 916.65 $LUKA
    │
    ├─> Transfer 28.35 $LUKA → [APP_REVENUE_COLLECTOR]
    │   └─> Apply Split 30/30/40
    │       ├─> 8.5 $LUKA → LP_POWER
    │       ├─> 8.5 $LUKA → O&M/Staking
    │       └─> 11.35 $LUKA → KASH
    │
    └─> Transfer 916.65 $LUKA → [SELLER_WALLET]
    │
    ↓
[RESULTADO FINAL]
    │
    ├─> Buyer paid: 1,000 $LUKA
    ├─> Protocol captured:
```

```
│   ├─> Stage Fee: 55 $LUKA
│   └─> Commission (30/30/40): 28.35 $LUKA
├─> Seller received: 916.65 $LUKA
│
└─> Total protocol revenue: 83.35 $LUKA (8.335%) ✓
```

FLUJO 5: Vesting del Vault Society (Jaguar Lock)


FLUJO: Activación y Distribución Jaguar Lock
[PYTH_ORACLE_MONITOR] (runs every 24h)
Query:
├─> BTC/USD price
├─> SOL/USD price
├─> USDC/USD price
└─> LINK/USD price
[VAULT_VALUE_CALCULATOR]
Calculate Total TVL:
VAULT_KASH_CORE + VAULT_KASH_SOCIETY
Example:
├─> BTC holdings: 15 BTC × $95,000 = $1,425,000
├─> SOL holdings: 120,000 SOL × $22 = $2,640,000
├─> USDC holdings: $800,000
└─> LINK holdings: 25,000 LINK × $14 = $350,000
TOTAL TVL = $5,215,000 USD ✓
[JAGUAR_LOCK_STATE_MACHINE]
Check Activation Conditions:
Condition 1: TVL >= $5,000,000 ✓
Condition 2: OR Month >= 13
IF ACTIVATED:
└─> Set: JAGUAR_LOCK_STATUS = ACTIVE
Set: ACTIVATION_TIMESTAMP = current_time
[VESTING_SCHEDULE_CALCULATOR]
Accumulated Capital at Activation:
├─> Genesis 30%: $X
├─> Accumulated Society Fees: $Y
└─> TOTAL LOCKED: $(X + Y)
Vesting Terms:
├─> Duration: 48 months
├─> Linear release: 1/48 per month
└─> Monthly unlock: $(X + Y) / 48
[MONTHLY_CLAIM_FUNCTION] (callable by authorized parties)

```
    │ Calculate Claimable Amount:
    │   Months_since_activation = (current_time - activation_time) / 30 days
    │   Vested_amount = (Total_locked / 48) * Months_since_activation
    │   Already_claimed = tracking_account.total_claimed
    │   Claimable = Vested_amount - Already_claimed
    │
    ├─> IF Claimable > 0:
    │   └─> Transfer from VAULT_KASH_SOCIETY to authorized wallets
    │       (Marketing, Team, Investors según distribución acordada)
    │
    └─> Update tracking_account.total_claimed += Claimable
    │
    ↓
[POST-ACTIVATION FLOW] (for new fees after activation)
    │
    │ Society Fees generated AFTER activation:
    │ └─> Distributed MONTHLY without vesting
    │     (Direct access, perpetual)
    │
    └─> Example: Month 14 Society fee = 100 $LUKA
        └─> Distributed immediately ✓
```

2.3 Wallets Temporales y de Procesamiento

BUFFER ACCOUNTS (Temporal Holding)

```
1. JUPITER_SWAP_TEMP_ACCOUNT
   └─> Lifetime: Single transaction
   └─> Purpose: Hold assets during atomic swap execution
   └─> Auto-closed after completion

2. MULTI_ASSET_CONVERSION_BUFFER
   └─> Holds: SOL/USDC/cBTC during fee collection
   └─> Converts to $LUKA in batches
   └─> Threshold: 20 SOL equivalent

3. LP_INJECTION_PREP_ACCOUNT
   └─> Holds: 50% $LUKA + 50% SOL equivalent
   └─> Purpose: Pair formation before LP add
   └─> Auto-transfer to Raydium/Orca

4. BURN_EXECUTION_ACCOUNT
   └─> Receives: $LUKA scheduled for burn
   └─> Executes: Transfer to burn address
   └─> Frequency: Triggered by threshold (20 SOL equiv)
```

---

3. ESPECIFICACIONES COMPLETAS DE SMART CONTRACTS

3.1 Módulos Core del Protocolo (Anchor/Rust)

MÓDULO 1: TRANSFER_HOOK_MANAGER

```
rust
```

```rust
// ============================================================================
// MÓDULO: TRANSFER_HOOK_MANAGER
// Propósito: Interceptar todas las transferencias de $LUKA y aplicar fees
// ============================================================================

use anchor_lang::prelude::*;
use anchor_spl::token_2022::{self, TransferChecked};

#[program]
pub mod transfer_hook_manager {
    use super::*;

    // ──────────────────────────────────────────────────────────────────────
    // INSTRUCTION: execute_transfer_with_fee
    // ──────────────────────────────────────────────────────────────────────
    pub fn execute_transfer_with_fee(
        ctx: Context<TransferWithFee>,
        amount: u64,
    ) -> Result<()> {

        // STEP 1: Determine Protocol Stage
        let protocol_state = &ctx.accounts.protocol_state;
        let current_stage = determine_stage(
            protocol_state.market_cap,
            protocol_state.circulating_supply,
            protocol_state.total_supply,
        )?;

        // STEP 2: Check Anti-Whale Tax
        let circulating_supply = protocol_state.circulating_supply;
        let whale_threshold = circulating_supply.checked_div(200).unwrap(); // 0.5%

        let fee_percentage = if amount > whale_threshold {
            2000 // 20% (basis points)
        } else {
            match current_stage {
                Stage::Genesis => 550,     // 5.5%
                Stage::Adoption => 350,    // 3.5%
                Stage::Expansion => 150,   // 1.5%
            }
        };

        // STEP 3: Check Anti-Bot Cooldown
        let user_state = &mut ctx.accounts.user_state;
        let current_time = Clock::get()?.unix_timestamp;

        require!(
            current_time - user_state.last_transfer_timestamp >= 60,
            ErrorCode::BotCooldownActive
        );

        user_state.last_transfer_timestamp = current_time;

        // STEP 4: Calculate Fee Distribution
        let fee_amount = amount
```

```
        .checked_mul(fee_percentage as u64)
        .unwrap()
        .checked_div(10000)
        .unwrap();

    let net_amount = amount.checked_sub(fee_amount).unwrap();

    let fee_split = calculate_fee_split(current_stage, fee_amount, amount > whale_threshold)?;

    // STEP 5: Execute Transfers
    // Transfer net amount to recipient
    token_2022::transfer_checked(
        CpiContext::new(
            ctx.accounts.token_program.to_account_info(),
            TransferChecked {
                from: ctx.accounts.sender_token_account.to_account_info(),
                to: ctx.accounts.recipient_token_account.to_account_info(),
                authority: ctx.accounts.sender.to_account_info(),
                mint: ctx.accounts.luka_mint.to_account_info(),
            },
        ),
        net_amount,
        ctx.accounts.luka_mint.decimals,
    )?;

    // Transfer fees to respective buffers
    distribute_fees(ctx, fee_split)?;

    // STEP 6: Emit event for analytics
    emit!(TransferEvent {
        sender: ctx.accounts.sender.key(),
        recipient: ctx.accounts.recipient.key(),
        gross_amount: amount,
        fee_amount,
        net_amount,
        stage: current_stage,
        timestamp: current_time,
    });

    Ok(())
}

// ═══════════════════════════════════════════════════════════════════════
// HELPER: determine_stage
// ═══════════════════════════════════════════════════════════════════════
fn determine_stage(
    market_cap: u64,
    circulating_supply: u64,
    total_supply: u64,
) -> Result<Stage> {

    let supply_percentage = circulating_supply
        .checked_mul(100)
        .unwrap()
        .checked_div(total_supply)
        .unwrap();
```

```
    if supply_percentage <= 33 {
        Ok(Stage::Expansion)
    } else if market_cap >= 50_000_000 {
        Ok(Stage::Adoption)
    } else {
        Ok(Stage::Genesis)
    }
}

// ─────────────────────────────────────────────────────────────────────────────
// HELPER: calculate_fee_split
// ─────────────────────────────────────────────────────────────────────────────
fn calculate_fee_split(
    stage: Stage,
    total_fee: u64,
    is_whale_penalty: bool,
) -> Result<FeeSplit> {

    if is_whale_penalty {
        // All excess goes to KASH
        return Ok(FeeSplit {
            kash: total_fee,
            lp_power: 0,
            staking: 0,
            om: 0,
        });
    }

    match stage {
        Stage::Genesis => {
            // 5.5% = 2.5% KASH + 1% LP + 1% Staking + 1% O&M
            Ok(FeeSplit {
                kash: total_fee.checked_mul(2500).unwrap().checked_div(5500).unwrap(),
                lp_power: total_fee.checked_mul(1000).unwrap().checked_div(5500).unwrap(),
                staking: total_fee.checked_mul(1000).unwrap().checked_div(5500).unwrap(),
                om: total_fee.checked_mul(1000).unwrap().checked_div(5500).unwrap(),
            })
        },
        Stage::Adoption => {
            // 3.5% = 1.5% KASH + 0.5% LP + 0.75% Staking + 0.75% O&M
            Ok(FeeSplit {
                kash: total_fee.checked_mul(1500).unwrap().checked_div(3500).unwrap(),
                lp_power: total_fee.checked_mul(500).unwrap().checked_div(3500).unwrap(),
                staking: total_fee.checked_mul(750).unwrap().checked_div(3500).unwrap(),
                om: total_fee.checked_mul(750).unwrap().checked_div(3500).unwrap(),
            })
        },
        Stage::Expansion => {
            // 1.5% = 0.5% KASH + 0.5% Staking + 0.5% O&M
            // Note: LP-Power handled separately in Stage 3 (30/40/30 rule)
            Ok(FeeSplit {
                kash: total_fee.checked_mul(500).unwrap().checked_div(1500).unwrap(),
                lp_power: 0,
                staking: total_fee.checked_mul(500).unwrap().checked_div(1500).unwrap(),
                om: total_fee.checked_mul(500).unwrap().checked_div(1500).unwrap(),
```

```
            })
        },
    }
}

#[derive(AnchorSerialize, AnchorDeserialize, Clone, Copy, PartialEq, Eq)]
pub enum Stage {
    Genesis,    // MC < $50M
    Adoption,   // MC >= $50M
    Expansion,  // Supply <= 33% of initial
}

#[derive(AnchorSerialize, AnchorDeserialize, Clone)]
pub struct FeeSplit {
    pub kash: u64,
    pub lp_power: u64,
    pub staking: u64,
    pub om: u64,
}
```

MÓDULO 2: KASH_VAULT_MANAGER

rust

```rust
// ═══════════════════════════════════════════════════════════════════════
// MÓDULO: KASH_VAULT_MANAGER
// Propósito: Gestionar el Vault KASH y procesamiento de fees acumulados
// ═══════════════════════════════════════════════════════════════════════

use anchor_lang::prelude::*;
use anchor_spl::token::{self, Token, TokenAccount, Mint, Transfer};

#[program]
pub mod kash_vault_manager {
    use super::*;

    // ─────────────────────────────────────────────────────────────────────
    // INSTRUCTION: process_accumulated_fees
    // Triggers when buffer reaches 20 SOL equivalent
    // ─────────────────────────────────────────────────────────────────────
    pub fn process_accumulated_fees(
        ctx: Context<ProcessFees>,
    ) -> Result<()> {

        let protocol_state = &ctx.accounts.protocol_state;
        let current_stage = protocol_state.current_stage;

        let buffer_balance = ctx.accounts.kash_fee_buffer.amount;

        // Verify threshold (20 SOL equivalent)
        let sol_price = get_pyth_price(&ctx.accounts.sol_price_feed)?;
        let luka_price = get_pyth_price(&ctx.accounts.luka_price_feed)?;

        let buffer_value_sol = calculate_sol_equivalent(
            buffer_balance,
            luka_price,
            sol_price,
        )?;

        require!(
            buffer_value_sol >= 20_000_000_000, // 20 SOL (9 decimals)
            ErrorCode::ThresholdNotMet
        );

        match current_stage {
            Stage::Genesis | Stage::Adoption => {
                // BURN the accumulated $LUKA
                burn_tokens(
                    ctx.accounts.kash_fee_buffer.to_account_info(),
                    ctx.accounts.luka_mint.to_account_info(),
                    ctx.accounts.token_program.to_account_info(),
                    buffer_balance,
                )?;

                msg!("Burned {} $LUKA from KASH buffer", buffer_balance);
            },

            Stage::Expansion => {
                // Execute 30/40/30 Split
```

```
            execute_stage3_distribution(ctx, buffer_balance)?;
        },
    }

    // Update protocol state
    let protocol_state = &mut ctx.accounts.protocol_state;
    protocol_state.last_fee_processing_timestamp = Clock::get()?.unix_timestamp;
    protocol_state.total_fees_processed += buffer_balance;

    Ok(())
}

// ═══════════════════════════════════════════════════════════════════════════
// INSTRUCTION: execute_stage3_distribution
// Implements the 30/40/30 rule for Stage 3
// ═══════════════════════════════════════════════════════════════════════════
pub fn execute_stage3_distribution(
    ctx: Context<ProcessFees>,
    total_amount: u64,
) -> Result<()> {

    // Calculate splits
    let lp_amount = total_amount.checked_mul(30).unwrap().checked_div(100).unwrap();
    let vault_core_amount = total_amount.checked_mul(40).unwrap().checked_div(100).unwrap();
    let society_amount = total_amount.checked_mul(30).unwrap().checked_div(100).unwrap();

    // 1. LP Injection (30%)
    // Swap 50% to SOL, pair with remaining 50% $LUKA, add liquidity
    let luka_for_lp = lp_amount.checked_div(2).unwrap();

    let sol_received = jupiter_swap_luka_to_sol(
        &ctx.accounts.jupiter_program,
        &ctx.accounts.kash_fee_buffer,
        luka_for_lp,
    )?;

    add_liquidity_and_burn(
        &ctx.accounts.raydium_program,
        luka_for_lp,
        sol_received,
        &ctx.accounts.lp_token_burn_vault,
    )?;

    // 2. Vault KASH Core (40%)
    distribute_to_kash_basket(
        ctx.accounts.vault_kash_core.to_account_info(),
        vault_core_amount,
        KashBasket {
            btc_percent: 45,
            sol_percent: 30,
            usdc_percent: 20,
            link_percent: 5,
        },
    )?;

    // 3. Vault KASH Society (30%)
```

```
    distribute_to_kash_basket(
        ctx.accounts.vault_kash_society.to_account_info(),
        society_amount,
        KashBasket {
            btc_percent: 45,
            sol_percent: 30,
            usdc_percent: 20,
            link_percent: 5,
        },
    )?;

    emit!(Stage3DistributionEvent {
        lp_injection: lp_amount,
        vault_core: vault_core_amount,
        vault_society: society_amount,
        timestamp: Clock::get()?.unix_timestamp,
    });

    Ok(())
}

// ═══════════════════════════════════════════════════════════════
// INSTRUCTION: distribute_to_kash_basket
// Swaps $LUKA to KASH assets (BTC/SOL/USDC/LINK)
// ═══════════════════════════════════════════════════════════════
pub fn distribute_to_kash_basket(
    vault_account: AccountInfo,
    total_luka: u64,
    basket: KashBasket,
) -> Result<()> {

    // Calculate amounts for each asset
    let btc_luka = total_luka.checked_mul(basket.btc_percent).unwrap().checked_div(100).unwrap();
    let sol_luka = total_luka.checked_mul(basket.sol_percent).unwrap().checked_div(100).unwrap();
    let usdc_luka = total_luka.checked_mul(basket.usdc_percent).unwrap().checked_div(100).unwrap();
    let link_luka = total_luka.checked_mul(basket.link_percent).unwrap().checked_div(100).unwrap();

    // Execute swaps via Jupiter with TWAP to minimize slippage
    jupiter_twap_swap_to_btc(btc_luka)?;
    jupiter_twap_swap_to_sol(sol_luka)?;
    jupiter_twap_swap_to_usdc(usdc_luka)?;
    jupiter_twap_swap_to_link(link_luka)?;

    msg!("Distributed {} $LUKA to KASH basket", total_luka);
    Ok(())
}

// ═══════════════════════════════════════════════════════════════
// INSTRUCTION: lp_power_autofeed
// Processes LP-Power buffer (Stages 1 & 2)
// ═══════════════════════════════════════════════════════════════
pub fn lp_power_autofeed(
    ctx: Context<ProcessLPPower>,
) -> Result<()> {

    let lp_buffer_balance = ctx.accounts.lp_power_buffer.amount;
```

```rust
        // Check threshold
        require!(
            lp_buffer_balance >= calculate_threshold()?,
            ErrorCode::ThresholdNotMet
        );

        // Swap 50% to SOL
        let luka_to_swap = lp_buffer_balance.checked_div(2).unwrap();
        let sol_received = jupiter_swap_luka_to_sol(
            &ctx.accounts.jupiter_program,
            &ctx.accounts.lp_power_buffer,
            luka_to_swap,
        )?;

        let remaining_luka = luka_to_swap; // The other 50%

        // Add liquidity to Raydium/Orca
        let lp_tokens_received = add_liquidity(
            &ctx.accounts.dex_program,
            remaining_luka,
            sol_received,
        )?;

        // BURN the LP tokens
        burn_lp_tokens(
            &ctx.accounts.lp_token_burn_vault,
            lp_tokens_received,
        )?;

        msg!("LP Power injection completed: {} $LUKA + {} SOL", remaining_luka, sol_received);
        Ok(())
    }
}

#[derive(AnchorSerialize, AnchorDeserialize, Clone)]
pub struct KashBasket {
    pub btc_percent: u8,   // 45
    pub sol_percent: u8,   // 30
    pub usdc_percent: u8,  // 20
    pub link_percent: u8,  // 5
}
```

MÓDULO 3: JAGUAR_LOCK_PROTOCOL

rust

```rust
// ═══════════════════════════════════════════════════════════════════════
// MÓDULO: JAGUAR_LOCK_PROTOCOL
// Propósito: Gestionar vesting del Vault Society
// ═══════════════════════════════════════════════════════════════════════

use anchor_lang::prelude::*;

#[program]
pub mod jaguar_lock_protocol {
    use super::*;

    // ─────────────────────────────────────────────────────────────────────
    // INSTRUCTION: check_activation_milestone
    // Runs daily via Clockwork automation
    // ─────────────────────────────────────────────────────────────────────
    pub fn check_activation_milestone(
        ctx: Context<CheckMilestone>,
    ) -> Result<()> {

        let jaguar_state = &mut ctx.accounts.jaguar_lock_state;

        // Skip if already activated
        if jaguar_state.is_activated {
            return Ok(());
        }

        // Calculate total vault TVL
        let vault_core_value = calculate_vault_value(
            &ctx.accounts.vault_kash_core,
            &ctx.accounts.price_feeds,
        )?;

        let vault_society_value = calculate_vault_value(
            &ctx.accounts.vault_kash_society,
            &ctx.accounts.price_feeds,
        )?;

        let total_tvl = vault_core_value
            .checked_add(vault_society_value)
            .unwrap();

        let current_time = Clock::get()?.unix_timestamp;
        let months_since_genesis = (current_time - jaguar_state.genesis_timestamp)
            .checked_div(30 * 24 * 60 * 60)
            .unwrap();

        // Check activation conditions
        let tvl_threshold = 5_000_000_000_000; // $5M with 6 decimals
        let month_threshold = 13;

        if total_tvl >= tvl_threshold || months_since_genesis >= month_threshold {

            // ACTIVATE JAGUAR LOCK
            jaguar_state.is_activated = true;
            jaguar_state.activation_timestamp = current_time;
```

```
        jaguar_state.total_locked_at_activation = vault_society_value;

        emit!(JaguarLockActivated {
            total_tvl,
            society_vault_value: vault_society_value,
            activation_timestamp: current_time,
            trigger: if total_tvl >= tvl_threshold { "TVL_THRESHOLD" } else { "MONTH_13" },
        });

        msg!("🐆 JAGUAR LOCK ACTIVATED! TVL: ${}, Society Vault: ${}",
             total_tvl / 1_000_000,
             vault_society_value / 1_000_000);
    }

    Ok(())
}

// ═══════════════════════════════════════════════════════════════════════════
// INSTRUCTION: claim_society_vesting
// Callable monthly by authorized parties
// ═══════════════════════════════════════════════════════════════════════════
pub fn claim_society_vesting(
    ctx: Context<ClaimVesting>,
    beneficiary: Pubkey,
) -> Result<()> {

    let jaguar_state = &ctx.accounts.jaguar_lock_state;
    let vesting_tracker = &mut ctx.accounts.vesting_tracker;

    require!(
        jaguar_state.is_activated,
        ErrorCode::JaguarLockNotActivated
    );

    let current_time = Clock::get()?.unix_timestamp;
    let months_since_activation = (current_time - jaguar_state.activation_timestamp)
        .checked_div(30 * 24 * 60 * 60)
        .unwrap();

    require!(
        months_since_activation > 0,
        ErrorCode::NoVestingAvailable
    );

    // Calculate vested amount (linear over 48 months)
    let total_locked = jaguar_state.total_locked_at_activation;
    let monthly_unlock = total_locked.checked_div(48).unwrap();

    let total_vested = monthly_unlock
        .checked_mul(std::cmp::min(months_since_activation, 48))
        .unwrap();

    let claimable = total_vested
        .checked_sub(vesting_tracker.total_claimed)
        .unwrap();
```

```
    require!(
        claimable > 0,
        ErrorCode::NothingToClaim
    );

    // Transfer from Vault Society to beneficiary
    // This would transfer actual KASH assets (BTC/SOL/USDC/LINK)
    transfer_kash_assets(
        &ctx.accounts.vault_kash_society,
        &ctx.accounts.beneficiary_wallet,
        claimable,
    )?;

    // Update tracker
    vesting_tracker.total_claimed = vesting_tracker
        .total_claimed
        .checked_add(claimable)
        .unwrap();
    vesting_tracker.last_claim_timestamp = current_time;

    emit!(VestingClaimed {
        beneficiary,
        amount: claimable,
        total_claimed: vesting_tracker.total_claimed,
        timestamp: current_time,
    });

    Ok(())
}

// ═══════════════════════════════════════════════════════════════════════════
// INSTRUCTION: distribute_post_activation_fees
// Distributes new Society fees generated after activation (no vesting)
// ═══════════════════════════════════════════════════════════════════════════
pub fn distribute_post_activation_fees(
    ctx: Context<DistributeSocietyFees>,
) -> Result<()> {

    let jaguar_state = &ctx.accounts.jaguar_lock_state;

    require!(
        jaguar_state.is_activated,
        ErrorCode::JaguarLockNotActivated
    );

    let current_month_fees = ctx.accounts.society_fee_buffer.amount;

    // Direct distribution (no vesting for post-activation fees)
    // Split among authorized beneficiaries
    distribute_to_beneficiaries(
        &ctx.accounts.society_fee_buffer,
        &ctx.accounts.beneficiary_accounts,
        current_month_fees,
    )?;

    emit!(PostActivationFeesDistributed {
```

```
            total_amount: current_month_fees,
            timestamp: Clock::get()?.unix_timestamp,
        });

        Ok(())
    }
}
```

MÓDULO 4: DUAL_FINANCIAL_ROUTER

rust

```rust
// ═══════════════════════════════════════════════════════════════════════
// MÓDULO: DUAL_FINANCIAL_ROUTER
// Propósito: Enrutar transacciones a Motor A o Motor B
// ═══════════════════════════════════════════════════════════════════════

use anchor_lang::prelude::*;

#[program]
pub mod dual_financial_router {
    use super::*;

    // ─────────────────────────────────────────────────────────────────
    // INSTRUCTION: route_transaction
    // Determines if transaction uses Motor A (Stage Fee) or Motor B (30/30/40)
    // ─────────────────────────────────────────────────────────────────
    pub fn route_transaction(
        ctx: Context<RouteTransaction>,
        transaction_type: TransactionType,
        amount: u64,
        input_asset: AssetType,
    ) -> Result<()> {

        match transaction_type {

            // MOTOR A: Stage Fee
            TransactionType::P2PTransfer |
            TransactionType::MarketPurchase |
            TransactionType::BettingDeposit |
            TransactionType::VacaDeposit => {

                // Check if multi-asset (requires double capture)
                if input_asset != AssetType::LUKA {
                    // LAYER 1: Swap fee
                    apply_stage_fee_to_swap(ctx, amount, input_asset)?;

                    // Execute swap to $LUKA
                    let luka_received = execute_swap_to_luka(ctx, amount, input_asset)?;

                    // LAYER 2: Transfer fee
                    apply_stage_fee_to_transfer(ctx, luka_received)?;

                } else {
                    // Single layer fee (already in $LUKA)
                    apply_stage_fee_to_transfer(ctx, amount)?;
                }
            },

            // MOTOR B: 30/30/40 Split
            TransactionType::ServicePayment |
            TransactionType::AIConsultation |
            TransactionType::EnergyPurchase |
            TransactionType::CNFTMinting |
            TransactionType::MarketplaceCommission |
            TransactionType::EventFee |
            TransactionType::BettingRake => {
```

```
            apply_service_split_30_30_40(ctx, amount)?;
        },
    }

    Ok(())
}

// ═══════════════════════════════════════════════════════════════════════
// HELPER: apply_service_split_30_30_40
// ═══════════════════════════════════════════════════════════════════════
fn apply_service_split_30_30_40(
    ctx: Context<RouteTransaction>,
    amount: u64,
) -> Result<()> {

    // 30% LP Power
    let lp_amount = amount.checked_mul(30).unwrap().checked_div(100).unwrap();
    transfer_to_buffer(
        ctx.accounts.source.to_account_info(),
        ctx.accounts.lp_power_buffer.to_account_info(),
        lp_amount,
    )?;

    // 30% O&M + Staking (split 50/50)
    let om_staking_total = amount.checked_mul(30).unwrap().checked_div(100).unwrap();
    let om_amount = om_staking_total.checked_div(2).unwrap();
    let staking_amount = om_staking_total.checked_sub(om_amount).unwrap();

    transfer_to_buffer(
        ctx.accounts.source.to_account_info(),
        ctx.accounts.om_wallet.to_account_info(),
        om_amount,
    )?;

    transfer_to_buffer(
        ctx.accounts.source.to_account_info(),
        ctx.accounts.staking_pool.to_account_info(),
        staking_amount,
    )?;

    // 40% KASH
    let kash_amount = amount.checked_mul(40).unwrap().checked_div(100).unwrap();
    transfer_to_buffer(
        ctx.accounts.source.to_account_info(),
        ctx.accounts.kash_fee_buffer.to_account_info(),
        kash_amount,
    )?;

    emit!(ServiceSplitEvent {
        lp_power: lp_amount,
        om: om_amount,
        staking: staking_amount,
        kash: kash_amount,
        timestamp: Clock::get()?.unix_timestamp,
    });
```

```
        Ok(())
    }
}

#[derive(AnchorSerialize, AnchorDeserialize, Clone, PartialEq, Eq)]
pub enum TransactionType {
    // Motor A (Stage Fee)
    P2PTransfer,
    MarketPurchase,
    BettingDeposit,
    VacaDeposit,

    // Motor B (30/30/40 Split)
    ServicePayment,
    AIConsultation,
    EnergyPurchase,
    CNFTMinting,
    MarketplaceCommission,
    EventFee,
    BettingRake,
}

#[derive(AnchorSerialize, AnchorDeserialize, Clone, PartialEq, Eq)]
pub enum AssetType {
    LUKA,
    SOL,
    USDC,
    cBTC,
    LINK,
}
```

MÓDULO 5: UNIVERSAL_FEE_EXTRACTOR

rust

```
// ═══════════════════════════════════════════════════════════════════
// MÓDULO: UNIVERSAL_FEE_EXTRACTOR
// Propósito: Capturar fees de cualquier activo SPL en la App
// ═══════════════════════════════════════════════════════════════════

use anchor_lang::prelude::*;
use anchor_spl::token::{self, Token, TokenAccount, Transfer};

#[program]
pub mod universal_fee_extractor {
    use super::*;

    // ─────────────────────────────────────────────────────────────
    // INSTRUCTION: extract_universal_fee
    // Monitors all SPL transfers in LUKASH App
    // ─────────────────────────────────────────────────────────────
    pub fn extract_universal_fee(
        ctx: Context<ExtractFee>,
        amount: u64,
        asset_type: AssetType,
    ) -> Result<()> {

        let protocol_state = &ctx.accounts.protocol_state;
        let current_stage = protocol_state.current_stage;

        // Get fee percentage based on stage
        let fee_percentage = match current_stage {
            Stage::Genesis => 550,     // 5.5%
            Stage::Adoption => 350,    // 3.5%
            Stage::Expansion => 150,   // 1.5%
        };

        let fee_amount = amount
            .checked_mul(fee_percentage as u64)
            .unwrap()
            .checked_div(10000)
            .unwrap();

// If asset is not $LUKA, swap fee portion to $LUKA
if asset_type != AssetType::LUKA {

    // Transfer fee to swap buffer
    transfer_to_swap_buffer(
        ctx.accounts.user_token_account.to_account_info(),
        ctx.accounts.swap_buffer.to_account_info(),
        fee_amount,
        asset_type,
    )?;

    // Execute swap to $LUKA via Jupiter
    let luka_received = jupiter_swap_to_luka(
        &ctx.accounts.jupiter_program,
        &ctx.accounts.swap_buffer,
        fee_amount,
        asset_type,
```

```
        )?;

        // Route $LUKA to distribution
        route_to_distribution(
            ctx.accounts.swap_buffer.to_account_info(),
            ctx.accounts.fee_accumulator.to_account_info(),
            luka_received,
            current_stage,
        )?;

    } else {
        // Direct transfer if already in $LUKA
        route_to_distribution(
            ctx.accounts.user_token_account.to_account_info(),
            ctx.accounts.fee_accumulator.to_account_info(),
            fee_amount,
            current_stage,
        )?;
    }

    emit!(UniversalFeeExtracted {
        user: ctx.accounts.user.key(),
        asset_type,
        original_amount: amount,
        fee_amount,
        luka_converted: if asset_type != AssetType::LUKA { Some(fee_amount) } else { None },
        stage: current_stage,
        timestamp: Clock::get()?.unix_timestamp,
    });

    Ok(())
  }
}
```

MÓDULO 6: ISO_20022_MEMO_INJECTOR

rust

```rust
// ═══════════════════════════════════════════════════════════════════════
// MÓDULO: ISO_20022_MEMO_INJECTOR
// Propósito: Adjuntar metadata XML ISO 20022 a todas las transacciones
// ═══════════════════════════════════════════════════════════════════════

use anchor_lang::prelude::*;

#[program]
pub mod iso_memo_injector {
    use super::*;

    // ─────────────────────────────────────────────────────────────────
    // INSTRUCTION: inject_iso_memo
    // Attaches ISO 20022 XML to Solana transaction memo
    // ─────────────────────────────────────────────────────────────────
    pub fn inject_iso_memo(
        ctx: Context<InjectMemo>,
        sender_id: String,
        receiver_id: String,
        amount: u64,
        purpose_code: String,
    ) -> Result<()> {

        let current_time = Clock::get()?.unix_timestamp;

        // Construct ISO 20022 XML string
        let iso_memo = format!(
            r#"<?xml version="1.0" encoding="UTF-8"?>
            <Document xmlns="urn:iso:std:iso:20022:tech:xsd:pain.001.001.09">
              <CstmrCdtTrfInitn>
                <GrpHdr>
                  <MsgId>{}</MsgId>
                  <CreDtTm>{}</CreDtTm>
                </GrpHdr>
                <PmtInf>
                  <Dbtr>
                    <Id>{}</Id>
                  </Dbtr>
                  <Cdtr>
                    <Id>{}</Id>
                  </Cdtr>
                  <InstdAmt Ccy="LUKA">{}</InstdAmt>
                  <Purp>
                    <Cd>{}</Cd>
                  </Purp>
                </PmtInf>
              </CstmrCdtTrfInitn>
            </Document>"#,
            ctx.accounts.transaction_id,
            current_time,
            sender_id,
            receiver_id,
            amount,
            purpose_code
        );
```

```
    // Attach to Solana transaction as memo
    // (This would use the Memo program in Solana)
    anchor_lang::solana_program::program::invoke(
        &spl_memo::build_memo(
            iso_memo.as_bytes(),
            &[ctx.accounts.payer.key],
        ),
        &[ctx.accounts.payer.to_account_info()],
    )?;

    emit!(ISOMemoAttached {
        transaction_id: ctx.accounts.transaction_id,
        sender: sender_id,
        receiver: receiver_id,
        amount,
        purpose: purpose_code,
        timestamp: current_time,
    });

    Ok(())
  }
}
```

3.2 Account Structures

rust

```rust
// ============================================================================
// ACCOUNT STRUCTURES
// ============================================================================

#[account]
pub struct ProtocolState {
    pub current_stage: Stage,
    pub market_cap: u64,                // En USD con 6 decimales
    pub circulating_supply: u64,
    pub total_supply: u64,
    pub total_burned: u64,
    pub last_fee_processing_timestamp: i64,
    pub total_fees_processed: u64,
    pub genesis_timestamp: i64,
    pub bump: u8,
}

#[account]
pub struct UserState {
    pub owner: Pubkey,
    pub jaguar_score: u32,
    pub total_transactions: u64,
    pub total_volume: u64,
    pub last_transfer_timestamp: i64,
    pub kyc_tier: u8,                   // 1, 2, or 3
    pub is_staking: bool,
    pub staked_amount: u64,
    pub bump: u8,
}

#[account]
pub struct JaguarLockState {
    pub is_activated: bool,
    pub genesis_timestamp: i64,
    pub activation_timestamp: i64,
    pub total_locked_at_activation: u64,
    pub vesting_duration_months: u8,    // 48
    pub bump: u8,
}

#[account]
pub struct VestingTracker {
    pub beneficiary: Pubkey,
    pub total_allocated: u64,
    pub total_claimed: u64,
    pub last_claim_timestamp: i64,
    pub bump: u8,
}

#[account]
pub struct FeeAccumulator {
    pub kash_buffer: u64,
    pub lp_power_buffer: u64,
    pub threshold_sol_equivalent: u64,  // 20 SOL
    pub last_processed: i64,
```

```
    pub bump: u8,
}
```

---

4. MODELO FINANCIERO Y PROYECCIONES

4.1 Supuestos del Modelo

Capital Inicial Estimado (Preventa 45%):

- Precio preventa: $0.0001 por $LUKA
- Tokens en preventa: 450,000,000 $LUKA
- **Capital recaudado: $45,000 USD**

Distribución Genesis del Capital ($45,000):

```
30% LP Injection    = $13,500  →  Pareado con 300M $LUKA y quemado
40% Vault KASH Core = $18,000  →  Convertido a BTC/SOL/USDC/LINK
30% Vault Society   = $13,500  →  Bloqueado bajo Jaguar Lock
```

Composición Inicial del Vault KASH ($31,500 total):

```
BTC (45%):  $14,175  ≈ 0.149 BTC @ $95,000/BTC
SOL (30%):  $9,450   ≈ 429 SOL @ $22/SOL
USDC (20%): $6,300   = 6,300 USDC
LINK (5%):  $1,575   ≈ 112.5 LINK @ $14/LINK
```

Supuestos de Crecimiento de Usuarios:

- Mes 1-3: 1,000-5,000 usuarios (early adopters)
- Mes 4-6: 10,000-30,000 usuarios (crecimiento viral)
- Mes 7-12: 50,000-150,000 usuarios (fase de adopción)
- Año 2: 300,000-1,000,000 usuarios (consolidación)

Supuestos de Transaccionalidad:

- Transacciones/usuario/mes: 15-30 (promedio 20)
- Valor promedio por transacción: $5-50 $LUKA
- Mix de transacciones:
    - 70% P2P/Marketplace (Motor A)
    - 30% Servicios App (Motor B)

4.2 Proyecciones Mensuales - Año 1

TABLA RESUMEN - PROYECCIONES AÑO 1 (Ver archivo Excel para detalles completos)

Mes	Usuarios	Precio $LUKA	Market Cap	Tokens Quemados	Vault TVL	Etapa
1	1,000	$0.000115	$80,500	300M	$32,000	Genesis
3	6,500	$0.000145	$101,500	305M	$38,000	Genesis
6	26,500	$0.000190	$133,000	318M	$65,000	Genesis
9	86,500	$0.000235	$164,500	335M	$145,000	Genesis
12	236,500	$0.000280	$196,000	358M	$285,000	Genesis

Proyección Base - Fin Año 1:

- Usuarios totales: ~240,000
- Market Cap: ~$200,000 USD
- Supply circulante: ~640M tokens (36% quemado)
- Vault TVL: ~$300,000 USD
- Precio $LUKA: ~$0.00028 (180% APY)

Notas Importantes:

1. Las proyecciones asumen crecimiento viral moderado (no incluyen eventos Black Swan positivos)
2. El Market Cap de $50M para Etapa Adoption podría alcanzarse en el Mes 18-24 con ejecución perfecta
3. El Vault TVL de $5M para Jaguar Lock probablemente se active por tiempo (mes 13) antes que por valor
4. La transición a Etapa Expansion (supply ≤33%) proyectada para meses 24-30

4.3 Análisis Detallado de Fees y Distribución

MOTOR A - FEE TRANSACCIONAL (70% del volumen)

```
Etapa Genesis (5.5% fee):
═══════════════════════════════════════════

Ejemplo: Transferencia de 1,000 $LUKA

Fee Total: 55 $LUKA
├─ 2.5% → KASH Buffer: 25 $LUKA → [BURN]
├─ 1.0% → LP Power: 10 $LUKA → [Inyección LP]
├─ 1.0% → Staking: 10 $LUKA → [Rewards stakers]
└─ 1.0% → O&M: 10 $LUKA → [Operational costs]

Usuario recibe: 945 $LUKA

Proyección Mensual (Mes 6):
- Volumen P2P/Marketplace: 2,000,000 $LUKA
- Fee capturado (70% × 5.5%): 77,000 $LUKA
  - KASH (quemado): 35,000 $LUKA
  - LP Power: 14,000 $LUKA
  - Staking: 14,000 $LUKA
  - O&M: 14,000 $LUKA
```

MOTOR B - SPLIT 30/30/40 (30% del volumen)

```
Servicios de la App:
═══════════════════════════════════════

Ejemplo: Compra de 100 Energy Credits = 50 $LUKA

Split 30/30/40:
├─ 30% → LP Power: 15 $LUKA → [Liquidez]
├─ 30% → O&M/Staking: 15 $LUKA → [50/50 split]
│   ├─ 7.5 $LUKA → O&M
│   └─ 7.5 $LUKA → Staking
└─ 40% → KASH: 20 $LUKA → [Conversión a activos duros]

Proyección Mensual (Mes 6):
- Volumen Servicios: 500,000 $LUKA
- Fee promedio servicios (5%): 25,000 $LUKA
  - LP Power: 7,500 $LUKA
  - O&M: 3,750 $LUKA
  - Staking: 3,750 $LUKA
  - KASH Vault: 10,000 $LUKA → $2.80 USD al vault
```

4.4 Proyección de Crecimiento del Vault KASH

COMPOSICIÓN DINÁMICA DEL VAULT

```
Estado Inicial (Día 0):
═══════════════════════════════════════

Capital: $31,500 USD (70% de $45K preventa)

BTC (45%): 0.149 BTC @ $95,000 = $14,175
SOL (30%): 429 SOL @ $22 = $9,450
USDC (20%): 6,300 USDC = $6,300
LINK (5%): 112.5 LINK @ $14 = $1,575

Estado Proyectado (Mes 12):
═══════════════════════════════════════

Capital acumulado: ~$285,000 USD

Fuentes de crecimiento:
1. Conversiones KASH de fees (Etapa 1-2): $0 (se quema)
2. Split 40% de servicios: ~$180,000
3. Apreciación de activos holdings:
   - BTC: $95K → $115K (+21%)
   - SOL: $22 → $32 (+45%)
   - LINK: $14 → $18 (+28%)
4. Stage 3 conversions (si se activa): Potencial +$100K

Proyección conservadora mes 12:
BTC: 0.40 BTC @ $115,000 = $46,000
SOL: 2,800 SOL @ $32 = $89,600
USDC: $115,000
LINK: 2,150 LINK @ $18 = $38,700

Total Vault: ~$289,300 USD
```

TRAYECTORIA HACIA $5M TVL (JAGUAR LOCK)

Escenario Base:

Año 1: $285K
Año 2: $1.2M (incluye Stage 3 + mayor volumen)
Año 3: $3.5M
Año 4: $5.8M ✓ MILESTONE ALCANZADO

Escenario Optimista:

Mes 16-18: $5M+ (requiere viral adoption + bull market)

Activación Más Probable:

MES 13 por criterio temporal (backup garantizado)

4.5 Análisis de Sensibilidad

IMPACTO DE VARIABLES CRÍTICAS

Variable	Cambio	Impacto en MC (Año 1)	Impacto en Vault	Impacto en Adopción
Usuarios +50%	360K	+45% ($290K)	+25% ($356K)	Etapa Adoption Mes 12
Precio Token +100%	$0.00056	+100% ($400K)	+15% ($328K)	Más atractivo staking
Tx/Usuario +50%	30/mes	+20% ($240K)	+40% ($400K)	Fees incrementan
Fee Adoption (3.5%) early	Mes 6	-15% ($170K)	-30% ($200K)	Menor quema
Bull Market BTC	+50%	Sin cambio directo	+35% ($385K)	Atrae usuarios
Volumen Servicios +50%	45% mix	+10% ($220K)	+60% ($456K)	Motor B crece

Conclusiones del Análisis:

1. El **volumen de transacciones** es el driver más importante (afecta fees y quema)
2. La **apreciación de activos del vault** (especialmente SOL/BTC) puede acelerar el milestone de $5M
3. El **mix de transacciones** (Motor A vs Motor B) afecta significativamente el crecimiento del vault

---

5. ANÁLISIS DE TRANSICIÓN DE FASES

5.1 Triggers y Condiciones de Cambio de Etapa

ETAPA 1 → ETAPA 2 (Genesis → Adoption)

Condición: Market Cap ≧ $50,000,000 USD

Cálculo del Milestone:
Supply circulante: ~650M tokens (asumiendo quema continua)
Precio requerido: $50M / 650M = $0.0769 por $LUKA

Desde precio inicial ($0.0001):
ROI requerido: 769x o +76,900%

Proyección:
- Escenario conservador: No alcanzado en Año 1
- Escenario base: Mes 18-24
- Escenario optimista: Mes 8-12

Variables críticas:
✓ Listing en CEX tier 2-3 (Bybit, KuCoin)
✓ Partnership estratégico (project nativo Solana)
✓ Viral marketing campaign success
✓ Bull market general cripto

Impacto del cambio:
Fee: 5.5% → 3.5%
├─ KASH: 2.5% → 1.5%
├─ LP Power: 1.0% → 0.5%
├─ Staking: 1.0% → 0.75%
└─ O&M: 1.0% → 0.75%

Beneficios:
+ Menor fricción transaccional
+ Mayor velocidad de adopción
+ Competitivo vs otros protocolos

Riesgos:
- Menor presión deflacionaria
- Requiere más volumen para mismo revenue

ETAPA 1/2 → ETAPA 3 (Genesis/Adoption → Expansion)

Condición: Supply circulante ≤ 33% del inicial

Cálculo del Milestone:
Supply inicial circulante: 700M tokens (excl. LP quemado)
Target: 700M × 0.33 = 231M tokens circulantes
Quema requerida: 469M tokens

Proyección de quema:
Mes 1-6: ~18M quemados (3M/mes promedio)
Mes 7-12: ~40M quemados (6.7M/mes - crecimiento)
Año 1 total: ~58M quemados

Quema faltante: 469M - 58M = 411M tokens
Tiempo estimado: 24-36 meses adicionales

Escenario Acelerado (Adoption temprano):
Si MC alcanza $50M en mes 12:
- Fee reduce a 3.5% pero volumen 10x
- Quema mensual aumenta a 15M tokens/mes
- Milestone alcanzable en mes 24-30

Cambio operativo CRÍTICO:

KASH fee: Ya NO se quema → Se convierte a activos

Regla 30/40/30:
30% → LP (deepens liquidity)
40% → Vault KASH Core (fortalece respaldo)
30% → Vault KASH Society (distribución mensual post-lock)

Impacto en Vault:
Crecimiento mensual proyectado (Stage 3):
- Base: +$50K/mes
- Optimista: +$200K/mes
- Path to $5M: 18-24 meses

ACTIVACIÓN JAGUAR LOCK

Condición Doble (OR lógico):

1. Vault TVL ≥ $5,000,000 USD (verificado vía Pyth)
   OR
2. Tiempo ≥ Mes 13 desde TGE

Escenario Más Probable: MES 13 (temporal backup)

Análisis de probabilidades:
- Vault alcanza $5M antes de mes 13: 15%
- Vault alcanza $5M mes 13-18: 25%
- Activación por tiempo (mes 13): 60%

Al activarse:

1. Snapshot del Vault Society:
   Capital acumulado = Genesis 30% + Fees acumulados

   Ejemplo Mes 13:
   - Genesis 30%: $13,500
   - Fees acumulados (13 meses): ~$55,000
   - TOTAL LOCKED: $68,500

2. Inicio de Vesting Lineal:
   Duración: 48 meses
   Monthly unlock: $68,500 / 48 = $1,427/mes

   Beneficiarios (ejemplo):
   - Team: 40% → $571/mes
   - Marketing: 35% → $500/mes
   - Early Investors: 25% → $356/mes

3. Fees Post-Activación:
   Todos los fees de Society DESPUÉS del mes 13:
   → Distribución DIRECTA mensual (sin vesting)
   → Perpetua mientras el protocolo funcione

5.2 Cronología Proyectada de Milestones

TIMELINE PROYECTADO - MILESTONES FINANCIEROS

DÍA 0 (TGE)

✓ Preventa completa: $45,000
✓ LP establecido y quemado: 300M tokens
✓ Vault KASH inicializado: $31,500
✓ Etapa: GENESIS activada
✓ Smart contracts deployed
✓ Token live en Raydium

MES 1-3 (Early Adoption)

□ Usuarios: 1K → 7K
□ App LUKASH v1.0 live (iOS/Android)
□ KYC Tier 1-2 functional
□ Primeros 10M tokens quemados
□ Vault: $32K → $42K
□ Market Cap: ~$100K

MES 4-6 (Viral Phase Inicio)

□ Usuarios: 7K → 30K
□ Guerrilla marketing campaigns
□ Influencer partnerships (Tier 100-1000)
□ Marketplace P2P active (1000+ listings)
□ Vault: $42K → $75K
□ Market Cap: ~$150K
□ MILESTONE: Break-even operacional O&M

MES 7-9 (Growth Acceleration)

□ Usuarios: 30K → 90K
□ CEX listings (Tier 3): MEXC, Gate.io
□ Jungle Arena fully gamified
□ 30M tokens quemados acumulado
□ Vault: $75K → $160K
□ Market Cap: ~$180K

MES 10-12 (Consolidación)

□ Usuarios: 90K → 240K
□ KYC Tier 3 + scoring crediticio
□ Preparación Fase 4 (DeFi/Lending)
□ 50M tokens quemados acumulado
□ Vault: $160K → $285K
□ Market Cap: ~$200K

MES 13 (JAGUAR LOCK ACTIVATION) ⭐

✓ Activación temporal garantizada
✓ Snapshot Vault Society: ~$70K
✓ Inicio vesting 48 meses
✓ Distribución mensual post-lock: $1,400+/mes
□ Vault Total: ~$310K

MES 14-18 (DeFi Launch)

□ Usuarios: 240K → 500K
□ Fase 4: Crowdfunding/Lending live
□ Oráculos RWA integrados (Chainlink)
□ Primeros proyectos financiados
□ Posible Stage 2 (Adoption) si bull market
□ Vault: $310K → $800K
□ Market Cap objetivo: $50M (optimista)

MES 19-24 (Scaling)

□ Usuarios: 500K → 1M

□ CEX Tier 2: Bybit, KuCoin
□ 150M tokens quemados acumulado
□ Possible Stage 3 (Expansion) activación
□ Vault: $800K → $2.5M
□ Market Cap objetivo: $50M+ (Adoption stage)

MES 25-36 (Año 2-3: Maturity)

□ Fase 5: Tarjeta global, DAO governance
□ Stage 3 consolidado (30/40/30 rule)
□ Vault objetivo: $5M+ (milestone TVL alcanzado)
□ Supply deflacionado: <400M tokens circulantes
□ Market Cap objetivo: $100M+ (Top 300)

5.3 Factores de Riesgo en Transiciones

RIESGOS DE ETAPA GENESIS

Riesgo	Probabilidad	Impacto	Mitigación
Falta de liquidez inicial	ALTO	CRÍTICO	LP profundo desde día 1, market making
Bots y snipers en TGE	ALTO	ALTO	Anti-bot cooldown, límites por wallet
Fee 5.5% demasiado alto	MEDIO	ALTO	Educar sobre deflación, comparar con competencia
Bajo volumen transaccional	MEDIO	CRÍTICO	Marketing agresivo, incentivos early adopters
Dump post-preventa	MEDIO	ALTO	Vesting para preventa, anti-whale tax

RIESGOS DE TRANSICIÓN A ADOPTION

Riesgo	Probabilidad	Impacto	Mitigación
MC estancado <$50M	ALTO	MEDIO	Triggers alternativos (tiempo/supply)
Reducción fee afecta revenue	MEDIO	MEDIO	Compensar con volumen, enfoque en Motor B
Competencia de otros protocolos	ALTO	ALTO	Diferenciación (KASH backing, ISO 20022)

RIESGOS DE TRANSICIÓN A EXPANSION

Riesgo	Probabilidad	Impacto	Mitigación
Quema insuficiente	MEDIO	ALTO	Stage 3 puede activarse también por TVL
Vault growth lento	MEDIO	CRÍTICO	Agresiva captura fees Motor B, DeFi yield
Cambio a 30/40/30 confunde	BAJO	MEDIO	Comunicación clara, dashboard transparente

6. RECOMENDACIONES ESTRATÉGICAS

6.1 Prioridades Técnicas Pre-Launch

CRÍTICO (Semana 1-2):

1. **Smart Contract Audit Completo**
    - Auditoría de Certik, OtterSec o Sec3
    - Focus en Transfer Hook y fee distribution logic
    - Verificar no hay exploits en threshold triggers
    - Budget: $15,000-25,000
2. **Testing Exhaustivo en Devnet**
    - Simular 10,000+ transacciones
    - Probar todos los edge cases:
        - Anti-whale tax activation
        - Anti-bot cooldown bajo carga
        - Threshold accumulation y batch processing
        - Jupiter integration failures (fallback)
    - Load testing: 100 tx/segundo
3. **Oracles Integration (Pyth Network)**
    - Configurar price feeds:
        - SOL/USD
        - BTC/USD
        - LINK/USD
    - Backup oracle: Switchboard
    - Update frequency: 30 segundos
4. **Jupiter SDK Integration**
    - TWAP configuration para swaps grandes
    - Slippage tolerance: 1% máximo
    - Fallback DEX: Orca si Raydium falla
    - Test en diferentes condiciones de liquidez

ALTO (Semana 3-4):

5. **Multisig Configuration**
    - Protocol Authority: 3/5 multisig
    - Signers: 2 founders + 2 advisors + 1 community
    - Timelock: 48 horas para cambios críticos
6. **LP Lock Mechanism**
    - Verificar burn de LP tokens es irreversible
    - Proof of burn on-chain
    - Comunicar claramente en docs
7. **Clockwork/cron Jobs Setup**
    - Daily: Check Jaguar Lock activation
    - Hourly: Monitor threshold accumulator
    - Weekly: Vault TVL calculation y publish

6.2 Estrategia de Lanzamiento Optimizada

FASE PRE-LAUNCH (-30 días)

```
Objetivo: 5,000 whitelist + $45K capital

═══════════════════════════════════════════

Semana 1-2: Content Seeding
├─ Publicar technical docs y whitepaper
├─ AMAs en Discord/Telegram (x3)
├─ Thread storm en X (daily)
└─ Pitch deck a VCs (target: $20K private)

Semana 3-4: Community Building
├─ Zealy campaigns (10,000 XP = whitelist spot)
├─ Collab con proyectos Solana afines
├─ KOL outreach (Tier 20+100)
└─ Pre-sale announcement (soft cap $25K, hard cap $45K)

Semana 5: Pre-sale Execution
├─ Tiered pricing:
│  ├─ Tier 1: $0.00008 (first $10K)
│  ├─ Tier 2: $0.00010 (next $20K)
│  └─ Tier 3: $0.00012 (last $15K)
├─ Min: $100, Max: $2,000 per wallet
├─ Vesting: 20% TGE, 80% linear 6 meses
└─ Target: 200-300 presale wallets

Budget allocation:
- Audit: $20K
- Marketing: $10K
- Development: $8K
- Legal/compliance: $5K
- Reserve: $2K
```

LAUNCH DAY (D-Day)

HORA 0:00 (UTC)

□ Smart contracts deployed
□ LP seeded: $13,500 + 300M $LUKA
□ Vault KASH funded: $18K Core + $13.5K Society
□ Token live on Raydium
□ Dexscreener/Birdeye tracking active

HORA 1:00-6:00

□ Announcement storm (X, Telegram, Discord)
□ Presale 20% unlock
□ First trades monitoring
□ Anti-bot systems verification
□ Chart watching party (Discord voice)

HORA 6:00-24:00

□ Jupiter aggregator listing
□ Dextools trending push
□ Influencer coordinated posts
□ First fee batch processed (verify on-chain)
□ Team AMA (Evening UTC)

DÍA 2-7

□ CoinGecko/CMC application (fast track)
□ CEX conversations (MEXC, Gate.io)
□ Daily content (memes, stats, burns)
□ App LUKASH beta access (first 1000 users)
□ Staking pool activation

DÍA 8-30

□ Full app launch (iOS + Android)
□ First marketplace listings
□ Jaguar cNFT airdrops (early adopters)
□ Partnerships announcements
□ First burn report (transparency)

6.3 KPIs y Métricas de Éxito

DASHBOARD PÚBLICO (Actualización en Tiempo Real)

Métricas On-Chain:

1. Supply Circulante (live)
2. Tokens Quemados (acumulado + tasa)
3. Vault TVL (valoración actual)
4. LP Depth (liquidez total)
5. Holders Count
6. 24h Volume
7. Fee Collection (diario/mensual)

Métricas Off-Chain:

8. App Downloads (iOS + Android)
9. DAU/MAU (usuarios activos)
10. Transacciones App (por tipo)
11. Jaguar Score promedio
12. NPS (Net Promoter Score)

URL: dashboard.lukash.io (público)
Tech: Dune Analytics + custom API

OBJETIVOS NUMÉRICOS POR TRIMESTRE

Q	Usuarios	Market Cap	Vault TVL	Daily Volume	Stage
Q1	30K	$150K	$75K	$15K	Genesis
Q2	100K	$500K	$250K	$80K	Genesis
Q3	300K	$2M	$800K	$350K	Adoption (opt.)
Q4	700K	$10M	$2.5M	$1.5M	Adoption
Q5	1.5M	$50M+	$5M+	$8M	Expansion

6.4 Plan de Contingencia

ESCENARIO: Bajo Volumen Transaccional

Trigger: <5,000 tx/día después de mes 2

Acciones:
1. Programa de rewards agresivo (gastar 5% community wallet)
2. Reducir temporalmente friction (UX improvements)
3. Campañas de referidos (50 $LUKA por amigo activo)
4. Partnership con merchants (cashback)
5. Gamification aumentada (daily quests)

ESCENARIO: Presión Vendedora Excesiva

Trigger: Precio -40% desde ATH en 7 días

Acciones:
1. Comunicación transparente (no ocultar)
2. Buy-back con O&M wallet (si presupuesto permite)
3. Anuncio de partnerships/desarrollo
4. Demostrar vault growth (transparency builds trust)
5. Staking rewards boost temporal (+50% APY)

ESCENARIO: Fallo de Oracle

Trigger: Pyth price feed falla >5 minutos

Acciones automáticas (código):
1. Fallback a Switchboard oracle
2. Si ambos fallan: freeze fee processing (acumular)
3. Alertar equipo vía Discord bot
4. Manual override solo con multisig
5. Resume automático cuando oracle vuelve

ESCENARIO: Exploit/Security Breach

Trigger: Actividad sospechosa detectada

Protocolo:
1. PAUSE contract (emergency multisig)
2. Comunicado público inmediato
3. Audit emergency (12-24h turnaround)
4. Snapshot de holders pre-incident
5. Plan de compensación si funds lost
6. Upgraded contract deployment
7. Migration path claro

6.5 Optimizaciones Técnicas Recomendadas

GAS OPTIMIZATION

```rust
// Recomendaciones para Anchor programs:

1. Use PDAs sin seeds innecesarios
2. Batch operations cuando sea posible
3. Cerrar cuentas temporales
4. Minimize CPI calls (cross-program invocations)
5. Use u64 en vez de u128 si rango suficiente
6. Compute budget optimization:
   - Request exact units needed
   - Monitor y tune per instruction

Savings target: 30-40% vs naïve implementation
```

SCALABILITY

```
Proyección carga (Año 2):
- 1M usuarios
- 50K transacciones/día
- 2,000 TPS peak

Infrastructure:
├─ RPC nodes: QuickNode/Helius (redundancy)
├─ Indexing: Helius webhooks + custom DB
├─ App backend: Serverless (AWS Lambda/Vercel)
└─ CDN: Cloudflare (global edge)

Monitoring:
- Sentry (error tracking)
- Datadog (infrastructure)
- Dune Analytics (on-chain)
```

6.6 Recomendaciones Finales

HÁGALO: ✅ Transparencia absoluta (dashboard público) ✅ Over-communicate el valor del KASH backing ✅ Community-first approach (escuchar feedback) ✅ Iteración rápida en UX de la app ✅ Partnerships estratégicos (Solana ecosystem) ✅ Educación continua (YouTube, threads, AMAs)

NO HÁGALO: ❌ Prometer price targets o "garantías" de retorno ❌ Ocultar información o problemas ❌ Depender de un solo DEX/CEX/oracle ❌ Ignorar seguridad por velocidad ❌ Cambiar tokenomics post-launch sin governance ❌ Usar fondos del Vault para marketing

---

CONCLUSIÓN

El proyecto LUKASH presenta una arquitectura técnica sólida con un modelo económico innovador (Reserva Fraccionaria Inversa). Las proyecciones financieras muestran que:

VIABLE EN ESCENARIO BASE:

- El sistema es autosostenible desde el mes 4-6
- El Vault crece orgánicamente con la actividad
- La deflación de supply es programática y verificable

REQUIERE EJECUCIÓN DISCIPLINADA:

- Marketing agresivo pero honesto
- Desarrollo técnico impecable (audits, testing)
- Community management activo
- Transparencia en todas las métricas

DIFERENCIADORES CLAVE:

1. **Backing Real:** No es promesa, es código
2. **Utilidad Multi-capa:** No solo hold, sino use
3. **Compliance:** ISO 20022 posiciona para institucional
4. **Sustainable:** No depende de ponzinomics

RIESGOS PRINCIPALES:

- Adopción inicial lenta (market timing)
- Competencia en Solana ecosystem
- Complejidad técnica (múltiples contratos interdependientes)

MILESTONE CRÍTICO: El MES 13 (Jaguar Lock activation) es el punto de inflexión donde el proyecto demuestra su capacidad de generar y retener valor real. Si se alcanza con métricas sanas (usuarios, volumen, TVL), el proyecto tiene fundamentos para escalar a Fases 4-5.

PRÓXIMOS PASOS INMEDIATOS:

1. Completar desarrollo de smart contracts (semana 1-2)
2. Auditoría de seguridad (semana 3-4)
3. Testing exhaustivo (semana 5-6)
4. Pre-sale campaign (semana 7-8)
5. TGE y App launch (semana 9)

Elaborado por: Expert Blockchain Analyst
Fecha: Enero 28, 2026
Versión: 2.0 - DEFINITIVA

Este documento es la única fuente de verdad para la implementación técnica y proyecciones financieras de LUKASH. Cualquier cambio debe ser versionado y comunicado a todos los stakeholders.